

Chapter 25 - IE32

IE32 is the 32-bit predecessor of IE64. It uses the same memory bus, sees the same MMIO map, and has the same fixed-length encoding. The differences are entirely about size and complexity: IE32 has half-width registers, fewer of them, no MMU, no FPU, and a simpler instruction set.

For programs that fit in 4 gigabytes of address space and do not need floating-point or virtual memory, IE32 is faster to write for and easier to read back.

25.1 Register file

IE32 names its 16 general-purpose registers with letters rather than numbers:

Register	Role
A	Accumulator. Default destination for most arithmetic.
X, Y, Z	Index registers. Used for addressing and counters.
B, C, D, E, F, G, H, S, T, U, V, W	General purpose.
PC	Program counter (32 bits, not addressable as a GPR).
SP	Stack pointer (32 bits, not addressable as a GPR).

Each register holds one 32-bit unsigned value. Signed operations interpret the same bits as two's-complement.

There is no zero register on IE32. Code that needs the constant zero loads it into a free register or uses the addressing mode's immediate form.

25.2 Instruction encoding

Every IE32 instruction is 8 bytes long:

```
byte 0 : opcode           (8 bits)
byte 1 : register field   (8 bits - low 4 bits = register index)
byte 2 : addressing mode  (8 bits)
byte 3 : reserved         (8 bits)
bytes 4-7 : operand       (32 bits)
```

The operand interpretation depends on the addressing mode. Word alignment is required for jump and call targets.

25.3 Addressing modes

Code	Mode	Operand meaning
0x00	Immediate	Operand is the literal value
0x01	Register direct	Operand is a register index
0x02	Register indirect	Operand is a register holding the address
0x03	Memory indirect	Operand is the address of an address (double indirection)

Code	Mode	Operand meaning
0x04	Direct	Operand is the memory address

For example, LDA #0x42 (immediate) loads 0x42 into A; LDA \$F0700 (direct) loads from address 0xF0700; LDA (X) (register indirect) loads from the address in X.

25.4 Instruction set

25.4.1 Data movement

LOAD / STORE are the generic forms. The letter-suffixed forms target specific registers:

Mnemonic	Operation
LDA/LDX/LDY/LDZ	Load A/X/Y/Z from operand
LDB-LDW, LDH/LDS/LDT	Load extended register
STA/STX/STY/STZ	Store A/X/Y/Z to operand
STB-STW, STH/STS/STT	Store extended register
INC reg	Increment a register (or memory in operand)
DEC reg	Decrement a register (or memory in operand)

25.4.2 Arithmetic and logic

ADD, SUB, MUL, DIV, MOD, AND, OR, XOR, NOT, SHL, SHR. Each takes a register operand and an addressing-mode operand; the result lands in the named register.

Example: ADD A, X adds the value in X to A. ADD A, #5 adds an immediate 5 to A.

25.4.3 Control flow

IE32 has no flags. Conditional jumps test the named register directly:

Mnemonic	Branches if
JMP	unconditional
JZ reg, addr	reg == 0
JNZ reg, addr	reg != 0
JGT reg, addr	reg > 0 (signed)
JGE reg, addr	reg >= 0 (signed)
JLT reg, addr	reg < 0 (signed)
JLE reg, addr	reg <= 0 (signed)

The signed sense of JGT/JGE/JLT/JLE matters: a value of 0xFFFFFFFF is treated as -1 by these instructions, so JLT A, addr branches when A has its high bit set.

A typical comparison sequence is:

```

SUB A, X      ; A = A - X
JGT A, label  ; branch if A > X

```

25.4.4 Subroutines

Mnemonic	Operation
JSR addr	Push PC + 8 to the stack, jump to addr
RTS	Pop a return address from the stack, jump there
PUSH reg	SP -= 4; [SP] = reg
POP reg	reg = [SP]; SP += 4

The stack grows downward. The reset SP is `STACK_START = 0x9F000`.

25.4.5 System

Mnemonic	Operation
NOP	No operation
HALT	Halt the CPU
SEI	Set the interrupt-enable flag
CLI	Clear the interrupt-enable flag
RTI	Return from interrupt
WAIT n	Wait n cycles

25.5 Interrupts and timer

IE32 has a single interrupt vector at memory `0x0004`. When an interrupt fires:

1. PC is pushed to the stack.
2. The interrupt-enable flag is cleared.
3. PC is loaded from `0x0004`.

RTI reverses the sequence: pops PC and re-enables interrupts.

The built-in timer counts down at one tick per CPU cycle. When the count reaches zero, it reloads from `TIMER_PERIOD (0xF0808)` and (if enabled) raises an interrupt. `TIMER_COUNT (0xF0804)` is the live count.

25.6 Reset and boot

On reset, `PC = 0x0000` and `SP = STACK_START = 0x9F000`. The conventional first instruction at `0x0000` is a `JMP` to `PROG_START = 0x1000`, where the image executor places loaded program code.

25.7 A small example

A complete IE32 program that emits "HI" to the terminal and halts:

```
.org 0x1000

start:
    LDX #0xF0700      ; X = TERM_OUT
    LDA #'H'
    STA (X)
    LDA #'I'
    STA (X)
    LDA #13
    STA (X)

    LDX #0xF07F0      ; TERM_SENTINEL
    LDA #0xDEAD
    STA (X)           ; halts the CPU

halt:
    JMP halt
```

25.8 What comes next

Chapter 26 leaves the IE-native processors behind and begins the heritage section: the 6502, the small 8-bit CPU that powered the home computers of the late 1970s and early 1980s. The 6502 has only three registers and a 64-kilobyte address space, but much of the software written for it is still beautiful to read, and Intuition Engine runs it natively.